



TRABAJO PRÁCTICO 6: COLECCIONES Y SISTEMA DE STOCK



Resolución Caso Práctico 1:

Paso 1

Enum CategoriaProducto: Define las categorías disponibles en el sistema de stock (Alimentos, Electrónica, Ropa, Hogar), cada una con una descripción asociada. Esto facilita la clasificación de productos y mejora la legibilidad del código.

Código:

```
public enum CategoriaProducto {  
    ALIMENTOS("Productos comestibles"),  
    ELECTRONICA("Dispositivos electrónicos"),  
    ROPA("Prendas de vestir"),  
    HOGAR("Artículos para el hogar");  
  
    private final String descripcion;  
  
    CategoriaProducto(String descripcion) {  
        this.descripcion = descripcion;  
    }  
  
    public String getDescripcion() {  
        return descripcion;  
    }  
}
```

Paso 2

Clase Producto: Representa cada producto del inventario con atributos (id, nombre, precio, cantidad y categoría). Se implementó el método mostrarInfo() para imprimir la información del producto en consola.

Código:

```
public class Producto {  
    private String id;  
    private String nombre;  
    private double precio;
```

```

private int cantidad;
private CategoriaProducto categoria;

// Constructor
public Producto(String id, String nombre, double precio, int cantidad, CategoriaProducto
categoria) {
    this.id = id;
    this.nombre = nombre;
    this.precio = precio;
    this.cantidad = cantidad;
    this.categoria = categoria;
}

// Getters y Setters
public String getId() { return id; }
public void setId(String id) { this.id = id; }

public String getNombre() { return nombre; }
public void setNombre(String nombre) { this.nombre = nombre; }

public double getPrecio() { return precio; }
public void setPrecio(double precio) { this.precio = precio; }

public int getCantidad() { return cantidad; }
public void setCantidad(int cantidad) { this.cantidad = cantidad; }

public CategoriaProducto getCategoria() { return categoria; }
public void setCategoria(CategoriaProducto categoria) { this.categoria = categoria; }

// Método mostrarInfo()
public void mostrarInfo() {
    System.out.println("ID: " + id);
    System.out.println("Nombre: " + nombre);
    System.out.println("Precio: $" + precio);
    System.out.println("Cantidad: " + cantidad);
    System.out.println("Categoría: " + categoria + " - " + categoria.getDescripcion());
}

```

```
        System.out.println("-----");
    }
}
```

Paso 3

Clase Inventario: Administra los productos mediante un ArrayList. Permite agregar, listar, buscar, eliminar y actualizar productos, además de filtrar por categoría, rango de precios y calcular el stock total. También obtiene el producto con mayor stock y muestra las categorías disponibles.

Código:

```
import java.util.ArrayList;

public class Inventario {
    private ArrayList<Producto> productos;

    // Constructor
    public Inventario() {
        productos = new ArrayList<>();
    }

    // Agregar producto
    public void agregarProducto(Producto p) {
        productos.add(p);
    }

    // Listar todos los productos
    public void listarProductos() {
        for (Producto p : productos) {
            p.mostrarInfo();
        }
    }

    // Buscar producto por ID
    public Producto buscarProductoPorId(String id) {
        for (Producto p : productos) {
```

```

        if (p.getId().equalsIgnoreCase(id)) {
            return p;
        }
    }
    return null; // si no lo encuentra
}

// Eliminar producto por ID
public boolean eliminarProducto(String id) {
    Producto encontrado = buscarProductoPorId(id);
    if (encontrado != null) {
        productos.remove(encontrado);
        return true;
    }
    return false;
}

// Actualizar stock de un producto
public boolean actualizarStock(String id, int nuevaCantidad) {
    Producto p = buscarProductoPorId(id);
    if (p != null) {
        p.setCantidad(nuevaCantidad);
        return true;
    }
    return false;
}

// Filtrar por categoría
public void filtrarPorCategoria(CategoriaProducto categoria) {
    for (Producto p : productos) {
        if (p.getCategoria() == categoria) {
            p.mostrarInfo();
        }
    }
}
}

```

```

// Total del stock
public int obtenerTotalStock() {
    int total = 0;
    for (Producto p : productos) {
        total += p.getCantidad();
    }
    return total;
}

// Producto con mayor stock
public Producto obtenerProductoConMayorStock() {
    if (productos.isEmpty()) return null;
    Producto mayor = productos.get(0);
    for (Producto p : productos) {
        if (p.getCantidad() > mayor.getCantidad()) {
            mayor = p;
        }
    }
    return mayor;
}

// Filtrar productos por precio
public void filtrarProductosPorPrecio(double min, double max) {
    for (Producto p : productos) {
        if (p.getPrecio() >= min && p.getPrecio() <= max) {
            p.mostrarInfo();
        }
    }
}

// Mostrar categorías disponibles
public void mostrarCategoriasDisponibles() {
    for (CategoriaProducto c : CategoriaProducto.values()) {
        System.out.println(c + " → " + c.getDescripcion());
    }
}

```

```
}
```

Paso 4

Clase Main: Se creó un inventario, se añadieron cinco productos de distintas categorías y se probaron todas las funcionalidades solicitadas: listar, buscar, filtrar, eliminar, actualizar stock, calcular stock total, obtener el producto con mayor stock, filtrar por rango de precio y mostrar categorías disponibles.

Código:

```
public class Main {  
    public static void main(String[] args) {  
  
        // Crear inventario  
        Inventario inventario = new Inventario();  
  
        // Crear 5 productos  
        Producto p1 = new Producto("P001", "Arroz", 1500, 20, CategoriaProducto.ALIMENTOS);  
        Producto p2 = new Producto("P002", "Notebook", 250000, 5, CategoriaProducto.ELECTRONICA);  
        Producto p3 = new Producto("P003", "Remera", 3500, 15, CategoriaProducto.ROPA);  
        Producto p4 = new Producto("P004", "Silla", 12000, 10, CategoriaProducto.HOGAR);  
        Producto p5 = new Producto("P005", "Celular", 180000, 8, CategoriaProducto.ELECTRONICA);  
  
        // Agregar productos al inventario  
        inventario.agregarProducto(p1);  
        inventario.agregarProducto(p2);  
        inventario.agregarProducto(p3);  
        inventario.agregarProducto(p4);  
        inventario.agregarProducto(p5);  
  
        // 2. Listar productos  
        System.out.println("=== LISTADO DE PRODUCTOS ===");  
        inventario.listarProductos();  
  
        // 3. Buscar producto por ID  
        System.out.println("\n=== BUSCAR PRODUCTO POR ID: P003 ===");
```

```

Producto buscado = inventario.buscarProductoPorId("P003");
if (buscado != null) buscado.mostrarInfo();

// 4. Filtrar productos por categoría
System.out.println("\n=== PRODUCTOS EN CATEGORÍA ELECTRONICA ===");
inventario.filtrarPorCategoria(CategoriaProducto.ELECTRONICA);

// 5. Eliminar producto
System.out.println("\n=== ELIMINAR PRODUCTO P001 ===");
inventario.eliminarProducto("P001");
inventario.listarProductos();

// 6. Actualizar stock
System.out.println("\n=== ACTUALIZAR STOCK P002 ===");
inventario.actualizarStock("P002", 12);
inventario.buscarProductoPorId("P002").mostrarInfo();

// 7. Mostrar total de stock
System.out.println("\n=== TOTAL STOCK ===");
System.out.println("Total: " + inventario.obtenerTotalStock());

// 8. Producto con mayor stock
System.out.println("\n=== PRODUCTO CON MAYOR STOCK ===");
inventario.obtenerProductoConMayorStock().mostrarInfo();

// 9. Filtrar productos por precio (1000 - 3000)
System.out.println("\n=== PRODUCTOS CON PRECIO ENTRE 1000 Y 3000 ===");
inventario.filtrarProductosPorPrecio(1000, 3000);

// 10. Mostrar categorías disponibles
System.out.println("\n=== CATEGORÍAS DISPONIBLES ===");
inventario.mostrarCategoriasDisponibles();
}
}

```

Capturas de la consola de NetBeans mostrando la ejecución:

Output - TP-6 (run)

```
run:
=== LISTADO DE PRODUCTOS ===
ID: P001
Nombre: Arroz
Precio: $1500.0
Cantidad: 20
Categoría: ALIMENTOS - Productos comestibles
-----
ID: P002
Nombre: Notebook
Precio: $250000.0
Cantidad: 5
Categoría: ELECTRONICA - Dispositivos electrónicos
-----
ID: P003
Nombre: Remera
Precio: $3500.0
Cantidad: 15
Categoría: ROPA - Prendas de vestir
-----
ID: P004
Nombre: Silla
Precio: $12000.0
Cantidad: 10
Categoría: HOGAR - Artículos para el hogar
-----
ID: P005
Nombre: Celular
Precio: $180000.0
Cantidad: 8
Categoría: ELECTRONICA - Dispositivos electrónicos
-----

=== BUSCAR PRODUCTO POR ID: P003 ===
ID: P003
Nombre: Remera
Precio: $3500.0
Cantidad: 15
Categoría: ROPA - Prendas de vestir
-----

=== PRODUCTOS EN CATEGORÍA ELECTRONICA ===
ID: P002
Nombre: Notebook
Precio: $250000.0
Cantidad: 5
Categoría: ELECTRONICA - Dispositivos electrónicos
-----
```

Output - TP-6 (run)

```
ID: P005
Nombre: Celular
Precio: $180000.0
Cantidad: 8
Categoría: ELECTRONICA - Dispositivos electrónicos
-----

=== ELIMINAR PRODUCTO P001 ===
ID: P002
Nombre: Notebook
Precio: $250000.0
Cantidad: 5
Categoría: ELECTRONICA - Dispositivos electrónicos
-----
ID: P003
Nombre: Remera
Precio: $3500.0
Cantidad: 15
Categoría: ROPA - Prendas de vestir
-----
ID: P004
Nombre: Silla
Precio: $12000.0
Cantidad: 10
Categoría: HOGAR - Artículos para el hogar
-----
ID: P005
Nombre: Celular
Precio: $180000.0
Cantidad: 8
Categoría: ELECTRONICA - Dispositivos electrónicos
-----

=== ACTUALIZAR STOCK P002 ===
ID: P002
Nombre: Notebook
Precio: $250000.0
Cantidad: 12
Categoría: ELECTRONICA - Dispositivos electrónicos
-----

=== TOTAL STOCK ===
Total: 45

=== PRODUCTO CON MAYOR STOCK ===
ID: P003
Nombre: Remera
Precio: $3500.0
Cantidad: 15
Categoría: ROPA - Prendas de vestir
-----

=== PRODUCTOS CON PRECIO ENTRE 1000 Y 3000 ===

=== CATEGORÍAS DISPONIBLES ===
ALIMENTOS ? Productos comestibles
ELECTRONICA ? Dispositivos electrónicos
ROPA ? Prendas de vestir
HOGAR ? Artículos para el hogar
BUILD SUCCESSFUL (total time: 0 seconds)
```

Resolución Ejercicio Propuesto 2: Biblioteca y Libros:

Paso 1

Clase Autor: Representa al autor de un libro con atributos de identificación, nombre y nacionalidad. Incluye el método mostrarInfo() para visualizar la información en consola.

Código:

```

public class Autor {
    private String id;
    private String nombre;
    private String nacionalidad;

    // Constructor
    public Autor(String id, String nombre, String nacionalidad) {
        this.id = id;
        this.nombre = nombre;
        this.nacionalidad = nacionalidad;
    }

    // Getters y Setters
    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public String getNacionalidad() { return nacionalidad; }
    public void setNacionalidad(String nacionalidad) { this.nacionalidad = nacionalidad; }

    // Método mostrarInfo
    public void mostrarInfo() {
        System.out.println("Autor: " + nombre + " (ID: " + id + ")");
        System.out.println("Nacionalidad: " + nacionalidad);
    }
}

```

Paso 2

Clase Libro: Representa a un libro con atributos de ISBN, título, año de publicación y un objeto Autor. Incluye el método `mostrarInfo()` que muestra la información del libro junto con la del autor.

Código:

```

public class Libro {

```

```

private String isbn;
private String titulo;
private int anioPublicacion;
private Autor autor;

// Constructor
public Libro(String isbn, String titulo, int anioPublicacion, Autor autor) {
    this.isbn = isbn;
    this.titulo = titulo;
    this.anioPublicacion = anioPublicacion;
    this.autor = autor;
}

// Getters y Setters
public String getIsbn() { return isbn; }
public void setIsbn(String isbn) { this.isbn = isbn; }

public String getTitulo() { return titulo; }
public void setTitulo(String titulo) { this.titulo = titulo; }

public int getAnioPublicacion() { return anioPublicacion; }
public void setAnioPublicacion(int anioPublicacion) { this.anioPublicacion =
anioPublicacion; }

public Autor getAutor() { return autor; }
public void setAutor(Autor autor) { this.autor = autor; }

// Método mostrarInfo()
public void mostrarInfo() {
    System.out.println("Título: " + titulo);
    System.out.println("ISBN: " + isbn);
    System.out.println("Año de publicación: " + anioPublicacion);
    autor.mostrarInfo();
    System.out.println("-----");
}
}

```

Paso 3

Clase Biblioteca: Representa una biblioteca con un nombre y una colección de libros. Permite agregar, listar, buscar y eliminar libros, obtener la cantidad total, filtrar por año de publicación y mostrar los autores disponibles.

Código:

```
import java.util.ArrayList;

public class Biblioteca {
    private String nombre;
    private ArrayList<Libro> libros;

    // Constructor
    public Biblioteca(String nombre) {
        this.nombre = nombre;
        this.libros = new ArrayList<>();
    }

    // Agregar libro
    public void agregarLibro(String isbn, String titulo, int anioPublicacion, Autor autor) {
        Libro nuevo = new Libro(isbn, titulo, anioPublicacion, autor);
        libros.add(nuevo);
    }

    // Listar todos los libros
    public void listarLibros() {
        if (libros.isEmpty()) {
            System.out.println("No hay libros en la biblioteca.");
        } else {
            for (Libro l : libros) {
                l.mostrarInfo();
            }
        }
    }
}
```

```
// Buscar libro por ISBN
public Libro buscarLibroPorIsbn(String isbn) {
    for (Libro l : libros) {
        if (l.getIsbn().equalsIgnoreCase(isbn)) {
            return l;
        }
    }
    return null;
}
```

```
// Eliminar libro por ISBN
public boolean eliminarLibro(String isbn) {
    Libro l = buscarLibroPorIsbn(isbn);
    if (l != null) {
        libros.remove(l);
        return true;
    }
    return false;
}
```

```
// Cantidad total de libros
public int obtenerCantidadLibros() {
    return libros.size();
}
```

```
// Filtrar libros por año de publicación
public void filtrarLibrosPorAnio(int anio) {
    for (Libro l : libros) {
        if (l.getAnioPublicacion() == anio) {
            l.mostrarInfo();
        }
    }
}
```

```
// Mostrar todos los autores de los libros
public void mostrarAutoresDisponibles() {
```

```

        System.out.println("=== AUTORES DISPONIBLES ===");
        for (Libro l : libros) {
            System.out.println(l.getAutor().getNombre() + " (" + l.getAutor().getNacionalidad()
+ ")");
        }
    }
}

```

Paso 4

Clase Main: Se creó una biblioteca llamada “Biblioteca Central”, tres autores y cinco libros asociados a ellos. Luego se listaron los libros, se buscó por ISBN, se filtraron por año de publicación, se eliminó un libro, se mostró la cantidad total de libros y finalmente se listaron los autores disponibles.

Código:

```

public class Main {
    public static void main(String[] args) {

        // 1. Crear la biblioteca
        Biblioteca biblioteca = new Biblioteca("Biblioteca Central");

        // 2. Crear autores
        Autor a1 = new Autor("A01", "Gabriel García Márquez", "Colombiana");
        Autor a2 = new Autor("A02", "J.K. Rowling", "Británica");
        Autor a3 = new Autor("A03", "Julio Cortázar", "Argentina");

        // 3. Agregar 5 libros a la biblioteca
        biblioteca.agregarLibro("ISBN001", "Cien Años de Soledad", 1967, a1);
        biblioteca.agregarLibro("ISBN002", "Harry Potter y la piedra filosofal", 1997, a2);
        biblioteca.agregarLibro("ISBN003", "Rayuela", 1963, a3);
        biblioteca.agregarLibro("ISBN004", "Harry Potter y la cámara secreta", 1998, a2);
        biblioteca.agregarLibro("ISBN005", "El amor en los tiempos del cólera", 1985, a1);

        // 4. Listar todos los libros
        System.out.println("=== LISTADO DE LIBROS ===");
    }
}

```

```

biblioteca.listarLibros();

// 5. Buscar un libro por ISBN
System.out.println("\n=== BUSCAR LIBRO ISBN002 ===");
Libro buscado = biblioteca.buscarLibroPorIsbn("ISBN002");
if (buscado != null) {
    buscado.mostrarInfo();
}

// 6. Filtrar libros por año
System.out.println("\n=== LIBROS DEL AÑO 1997 ===");
biblioteca.filtrarLibrosPorAnio(1997);

// 7. Eliminar un libro
System.out.println("\n=== ELIMINAR LIBRO ISBN003 ===");
biblioteca.eliminarLibro("ISBN003");
biblioteca.listarLibros();

// 8. Mostrar cantidad de libros
System.out.println("\n=== TOTAL DE LIBROS EN LA BIBLIOTECA ===");
System.out.println("Cantidad: " + biblioteca.obtenerCantidadLibros());

// 9. Mostrar autores disponibles
biblioteca.mostrarAutoresDisponibles();
}
}

```

Capturas de la consola de NetBeans mostrando la ejecución:

```

=== LISTADO DE LIBROS ===
Título: Cien Años de Soledad
ISBN: ISBN001
Año de publicación: 1967
Autor: Gabriel García Márquez (ID: A01)
Nacionalidad: Colombiana
-----
Título: Harry Potter y la piedra filosofal
ISBN: ISBN002
Año de publicación: 1997
Autor: J.K. Rowling (ID: A02)
Nacionalidad: Británica
-----
Título: Rayuela
ISBN: ISBN003
Año de publicación: 1963
Autor: Julio Cortázar (ID: A03)
Nacionalidad: Argentina
-----
Título: Harry Potter y la cámara secreta
ISBN: ISBN004
Año de publicación: 1998
Autor: J.K. Rowling (ID: A02)
Nacionalidad: Británica
-----
Título: El amor en los tiempos del cólera
ISBN: ISBN005
Año de publicación: 1985
Autor: Gabriel García Márquez (ID: A01)
Nacionalidad: Colombiana
-----

=== BUSCAR LIBRO ISBN002 ===
Título: Harry Potter y la piedra filosofal
ISBN: ISBN002
Año de publicación: 1997
Autor: J.K. Rowling (ID: A02)
Nacionalidad: Británica
-----

=== LIBROS DEL AÑO 1997 ===
Título: Harry Potter y la piedra filosofal
ISBN: ISBN002
Año de publicación: 1997
Autor: J.K. Rowling (ID: A02)
Nacionalidad: Británica
-----

=== ELIMINAR LIBRO ISBN003 ===
Título: Cien Años de Soledad
ISBN: ISBN001
Año de publicación: 1967
Autor: Gabriel García Márquez (ID: A01)
Nacionalidad: Colombiana
-----
Título: Harry Potter y la piedra filosofal
ISBN: ISBN002
Año de publicación: 1997
Autor: J.K. Rowling (ID: A02)
Nacionalidad: Británica
-----
Título: Harry Potter y la cámara secreta
ISBN: ISBN004
Año de publicación: 1998
Autor: J.K. Rowling (ID: A02)
Nacionalidad: Británica
-----
Título: El amor en los tiempos del cólera
ISBN: ISBN005
Año de publicación: 1985
Autor: Gabriel García Márquez (ID: A01)
Nacionalidad: Colombiana
-----

=== TOTAL DE LIBROS EN LA BIBLIOTECA ===
Cantidad: 4
=== AUTORES DISPONIBLES ===
Gabriel García Márquez (Colombiana)
J.K. Rowling (Británica)
J.K. Rowling (Británica)
Gabriel García Márquez (Colombiana)
BUILD SUCCESSFUL (total time: 0 seconds)

```

Resolución Ejercicio: Universidad, Profesor y Curso (bidireccional 1 a N):

Paso 1

Clase Profesor: Representa un profesor con id, nombre y especialidad. Mantiene una lista de cursos que dicta y asegura la coherencia

bidireccional al agregar o eliminar cursos. Incluye métodos para listar cursos y mostrar información.

Código:

```
import java.util.ArrayList;
import java.util.List;

public class Profesor {
    private String id;
    private String nombre;
    private String especialidad;
    private List<Curso> cursos;

    public Profesor(String id, String nombre, String especialidad) {
        this.id = id;
        this.nombre = nombre;
        this.especialidad = especialidad;
        this.cursos = new ArrayList<>();
    }

    // Getters
    public String getId() { return id; }
    public String getNombre() { return nombre; }
    public String getEspecialidad() { return especialidad; }
    public List<Curso> getCursos() { return cursos; }

    // Agregar curso con sincronización segura
    public void agregarCurso(Curso c) {
        if (!cursos.contains(c)) {
            cursos.add(c);
        }
        if (c.getProfesor() != this) {
            c.setProfesor(this);
        }
    }
}
```

```

// Eliminar curso con sincronización segura
public void eliminarCurso(Curso c) {
    if (cursos.contains(c)) {
        cursos.remove(c);
    }
    if (c.getProfesor() == this) {
        c.setProfesor(null);
    }
}

// Métodos internos sin recursividad (usados por Curso)
public void agregarCursoSinRecursividad(Curso c) {
    if (!cursos.contains(c)) {
        cursos.add(c);
    }
}

public void eliminarCursoSinRecursividad(Curso c) {
    cursos.remove(c);
}

// Listar cursos
public void listarCursos() {
    if (cursos.isEmpty()) {
        System.out.println(nombre + " no dicta cursos.");
    } else {
        System.out.println("Cursos de " + nombre + ":");
        for (Curso c : cursos) {
            System.out.println("- " + c.getCodigo() + " : " + c.getNombre());
        }
    }
}

// Mostrar info
public void mostrarInfo() {
    System.out.println("Profesor: " + nombre + " (ID: " + id + ")");
}

```

```

        System.out.println("Especialidad: " + especialidad);
        System.out.println("Cantidad de cursos: " + cursos.size());
        System.out.println("-----");
    }
}

```

Paso 2

Clase Curso: Representa un curso con código y nombre. Tiene un profesor responsable y asegura la coherencia bidireccional al asignar o cambiar el profesor mediante `setProfesor()`. Incluye el método `mostrarInfo()` para mostrar datos del curso y del profesor.

Código:

```

public class Curso {
    private String codigo;
    private String nombre;
    private Profesor profesor;

    public Curso(String codigo, String nombre) {
        this.codigo = codigo;
        this.nombre = nombre;
        this.profesor = null;
    }

    // Getters
    public String getCodigo() { return codigo; }
    public String getNombre() { return nombre; }
    public Profesor getProfesor() { return profesor; }

    // Asignar/cambiar profesor con sincronización segura
    public void setProfesor(Profesor p) {
        if (this.profesor == p) return; // ya tiene ese profesor, no hacer nada

        Profesor anterior = this.profesor; // guardamos profesor anterior
        this.profesor = p;
    }
}

```

```

        if (anterior != null) {
            anterior.eliminarCursoSinRecursividad(this); // quitar del profesor anterior
        }

        if (p != null && !p.getCursos().contains(this)) {
            p.agregarCursoSinRecursividad(this); // agregar al nuevo profesor
        }
    }

    // Mostrar información
    public void mostrarInfo() {
        System.out.println("Curso: " + nombre + " (Código: " + codigo + ")");
        if (profesor != null) {
            System.out.println("Profesor: " + profesor.getNombre());
        } else {
            System.out.println("Profesor: No asignado");
        }
        System.out.println("-----");
    }
}

```

Paso 3

Clase Universidad: Representa la universidad con listas de profesores y cursos. Permite agregar, listar, buscar y eliminar profesores y cursos. Mantiene la coherencia de la relación bidireccional entre cursos y profesores al asignar, cambiar o eliminar.

Código:

```

import java.util.ArrayList;
import java.util.List;

public class Universidad {
    private String nombre;
    private List<Profesor> profesores;
    private List<Curso> cursos;

```

```

public Universidad(String nombre) {
    this.nombre = nombre;
    this.profesores = new ArrayList<>();
    this.cursos = new ArrayList<>();
}

// Agregar profesor
public void agregarProfesor(Profesor p) {
    profesores.add(p);
}

// Agregar curso
public void agregarCurso(Curso c) {
    cursos.add(c);
}

// Buscar profesor por ID
public Profesor buscarProfesorPorId(String id) {
    for (Profesor p : profesores) {
        if (p.getId().equalsIgnoreCase(id)) return p;
    }
    return null;
}

// Buscar curso por código
public Curso buscarCursoPorCodigo(String codigo) {
    for (Curso c : cursos) {
        if (c.getCodigo().equalsIgnoreCase(codigo)) return c;
    }
    return null;
}

// Asignar profesor a un curso
public void asignarProfesorACurso(String codigoCurso, String idProfesor) {
    Curso c = buscarCursoPorCodigo(codigoCurso);
    Profesor p = buscarProfesorPorId(idProfesor);
}

```

```

        if (c != null && p != null) {
            c.setProfesor(p);
        }
    }

// Listar profesores
public void listarProfesores() {
    for (Profesor p : profesores) {
        p.mostrarInfo();
        p.listarCursos();
    }
}

// Listar cursos
public void listarCursos() {
    for (Curso c : cursos) {
        c.mostrarInfo();
    }
}

// Eliminar curso
public boolean eliminarCurso(String codigo) {
    Curso c = buscarCursoPorCodigo(codigo);
    if (c != null) {
        if (c.getProfesor() != null) {
            c.getProfesor().eliminarCurso(c);
        }
        cursos.remove(c);
        return true;
    }
    return false;
}

// Eliminar profesor
public boolean eliminarProfesor(String id) {
    Profesor p = buscarProfesorPorId(id);

```

```

        if (p != null) {
            for (Curso c : new ArrayList<>(p.getCursos())) {
                c.setProfesor(null); // rompe relación bidireccional
            }
            profesores.remove(p);
            return true;
        }
        return false;
    }
}

```

Paso 4

Clase Main: Se creó una universidad, tres profesores y cinco cursos. Se asignaron profesores a cursos, se listaron los cursos y los profesores con sus cursos, se cambió un profesor, se eliminaron un curso y un profesor, y se verificó la sincronización bidireccional. Finalmente, se mostró la cantidad de cursos por profesor.

Código:

```

public class Main {

    public static void main(String[] args) {

        // 1. Crear universidad
        Universidad uni = new Universidad("Universidad Nacional");

        // 2. Crear profesores
        Profesor prof1 = new Profesor("P001", "Ana López", "Matemática");
        Profesor prof2 = new Profesor("P002", "Carlos Pérez", "Física");
        Profesor prof3 = new Profesor("P003", "Lucía Martínez", "Programación");

        // 3. Crear cursos
        Curso c1 = new Curso("C101", "Álgebra I");
        Curso c2 = new Curso("C102", "Física General");
        Curso c3 = new Curso("C103", "Programación I");
        Curso c4 = new Curso("C104", "Cálculo");
        Curso c5 = new Curso("C105", "Estructuras de Datos");
    }
}

```

```

// Agregar profesores y cursos a la universidad
uni.agregarProfesor(prof1);
uni.agregarProfesor(prof2);
uni.agregarProfesor(prof3);

uni.agregarCurso(c1);
uni.agregarCurso(c2);
uni.agregarCurso(c3);
uni.agregarCurso(c4);
uni.agregarCurso(c5);

// 4. Asignar profesores a cursos
uni.asignarProfesorACurso("C101", "P001");
uni.asignarProfesorACurso("C102", "P002");
uni.asignarProfesorACurso("C103", "P003");
uni.asignarProfesorACurso("C104", "P001");
uni.asignarProfesorACurso("C105", "P003");

// 5. Listar cursos con profesor y profesores con sus cursos
System.out.println("=== LISTA DE CURSOS ===");
uni.listarCursos();

System.out.println("\n=== LISTA DE PROFESORES ===");
uni.listarProfesores();

// 6. Cambiar profesor de un curso
System.out.println("\n=== CAMBIAR PROFESOR DEL CURSO C105 A P002 ===");
uni.asignarProfesorACurso("C105", "P002");

System.out.println("\n=== LISTA DE CURSOS DESPUÉS DEL CAMBIO ===");
uni.listarCursos();

System.out.println("\n=== LISTA DE PROFESORES DESPUÉS DEL CAMBIO ===");
uni.listarProfesores();

```



```

// 7. Eliminar un curso y un profesor
System.out.println("\n=== ELIMINAR CURSO C102 ===");
uni.eliminarCurso("C102");
uni.listarCursos();
uni.listarProfesores();

System.out.println("\n=== ELIMINAR PROFESOR P001 ===");
uni.eliminarProfesor("P001");
uni.listarCursos();
uni.listarProfesores();

// 8. Reporte: cantidad de cursos por profesor
System.out.println("\n=== CANTIDAD DE CURSOS POR PROFESOR ===");
for (Profesor p : new Profesor[]{prof1, prof2, prof3}) {
    System.out.println(p.getNombre() + ": " + p.getCursos().size() + " cursos");
}
}
}

```

Capturas de la consola de NetBeans mostrando la ejecución:

```

Main.java [-/Al x
Output - TP-6 (run)

run:
=== LISTA DE CURSOS ===
Curso: Álgebra I (Código: C101)
Profesor: Ana López
-----
Curso: Física General (Código: C102)
Profesor: Carlos Pérez
-----
Curso: Programación I (Código: C103)
Profesor: Lucía Martínez
-----
Curso: Cálculo (Código: C104)
Profesor: Ana López
-----
Curso: Estructuras de Datos (Código: C105)
Profesor: Lucía Martínez
-----

=== LISTA DE PROFESORES ===
Profesor: Ana López (ID: P001)
Especialidad: Matemática
Cantidad de cursos: 2
-----
Cursos de Ana López:
- C101 : Álgebra I
- C104 : Cálculo
Profesor: Carlos Pérez (ID: P002)
Especialidad: Física
Cantidad de cursos: 1
-----
Cursos de Carlos Pérez:
- C102 : Física General
Profesor: Lucía Martínez (ID: P003)
Especialidad: Programación
Cantidad de cursos: 2
-----
Cursos de Lucía Martínez:
- C103 : Programación I
- C105 : Estructuras de Datos

=== CAMBIAR PROFESOR DEL CURSO C105 A P002 ===

```

```

Output - TP-6 (run)

=== LISTA DE CURSOS DESPUÉS DEL CAMBIO ===
Curso: Álgebra I (Código: C101)
Profesor: Ana López
-----
Curso: Física General (Código: C102)
Profesor: Carlos Pérez
-----
Curso: Programación I (Código: C103)
Profesor: Lucía Martínez
-----
Curso: Cálculo (Código: C104)
Profesor: Ana López
-----
Curso: Estructuras de Datos (Código: C105)
Profesor: Carlos Pérez
-----

=== LISTA DE PROFESORES DESPUÉS DEL CAMBIO ===
Profesor: Ana López (ID: P001)
Especialidad: Matemática
Cantidad de cursos: 2
-----
Cursos de Ana López:
- C101 : Álgebra I
- C104 : Cálculo
Profesor: Carlos Pérez (ID: P002)
Especialidad: Física
Cantidad de cursos: 2
-----
Cursos de Carlos Pérez:
- C102 : Física General
- C105 : Estructuras de Datos
Profesor: Lucía Martínez (ID: P003)
Especialidad: Programación
Cantidad de cursos: 1
-----
Cursos de Lucía Martínez:
- C103 : Programación I

```

```

=== ELIMINAR CURSO C102 ===
Curso: Álgebra I (Código: C101)
Profesor: Ana López
-----
Curso: Programación I (Código: C103)
Profesor: Lucía Martínez
-----
Curso: Cálculo (Código: C104)
Profesor: Ana López
-----
Curso: Estructuras de Datos (Código: C105)
Profesor: Carlos Pérez
-----
Profesor: Ana López (ID: P001)
Especialidad: Matemática
Cantidad de cursos: 2
-----
Cursos de Ana López:
- C101 : Álgebra I
- C104 : Cálculo
Profesor: Carlos Pérez (ID: P002)
Especialidad: Física
Cantidad de cursos: 1
-----
Cursos de Carlos Pérez:
- C105 : Estructuras de Datos
Profesor: Lucía Martínez (ID: P003)
Especialidad: Programación
Cantidad de cursos: 1
-----
Cursos de Lucía Martínez:
- C103 : Programación I

```

```

=== ELIMINAR PROFESOR P001 ===
Curso: Álgebra I (Código: C101)
Profesor: No asignado
-----
Curso: Programación I (Código: C103)
Profesor: Lucía Martínez
-----
Curso: Cálculo (Código: C104)
Profesor: No asignado
-----
Curso: Estructuras de Datos (Código: C105)
Profesor: Carlos Pérez
-----
Profesor: Carlos Pérez (ID: P002)
Especialidad: Física
Cantidad de cursos: 1
-----
Cursos de Carlos Pérez:
- C105 : Estructuras de Datos
Profesor: Lucía Martínez (ID: P003)
Especialidad: Programación
Cantidad de cursos: 1
-----
Cursos de Lucía Martínez:
- C103 : Programación I

=== CANTIDAD DE CURSOS POR PROFESOR ===
Ana López: 0 cursos
Carlos Pérez: 1 cursos
Lucía Martínez: 1 cursos
BUILD SUCCESSFUL (total time: 0 seconds)

```

Link a mi repositorio de GitHub:

<https://github.com/NazarenoAranda/UTN-TUPaD-P2-Nazareno-Aranda/tree/master/UTN-TUPaD-P2-Nazareno-Aranda>

Nazareno Aranda